



CK · DIGITAL CONSTRUCTION

// DEPLOY AUTOMATION & COMPLIANCE

The Digital Delivery Toolkit

The Python automation patterns, Power Automate workflows and ISO 19650 verification checks used to manage digital delivery on the world's largest infrastructure programmes.

BIM Automation

ISO 19650

Model Health

Clash & Compliance

Christos Kofidis

Chartered Senior BIM Manager · MCI0B

ckofidis-git.github.io/ck-bim-site

ISO 19650 Pre-Submission Checklist

Run these checks on every information container before it leaves the CDE. Generic, standard-aligned best practice — adapt the codes to your project's information standard.

— A • Identification & Naming

- ☐ **Naming convention** — fields, separators and codes match the agreed standard; no spaces or illegal characters.
- ☐ **Revision** — container revision incremented; superseded versions withdrawn from the work-in-progress area.
- ☐ **Discipline / type / role** codes correct for the authoring task team.

— B • Status & Suitability

- ☐ **Suitability code** (S0–S7 / A–B) set for the intended use — shared, published or for-construction.
- ☐ **Review state** reflects the actual approval stage, not the aspirational one.
- ☐ **Authorization** captured before any container moves to the published state.

— C • Metadata & Classification

- ☐ **Classification** (e.g. Uniclass) applied to all required elements.
- ☐ **Mandatory property sets** present and populated — zero nulls in required fields.
- ☐ **Units, coordinate system and shared origin** verified against the project standard.

— D • Geometry & Coordination

- ☐ **Federation** — model aligns to the shared origin with no offset or rotation error.
- ☐ **Clash detection** run; hard clashes resolved or logged with an owner and status.
- ☐ **Hygiene** — no orphaned or duplicated elements; unused families and styles purged.

— E • CDE & Delivery

- ☐ **Workflow state** — container in the correct CDE state with a complete audit trail.
- ☐ **MIDP / TIDP** — deliverable matches the planned milestone entry.
- ☐ **Transmittal** recorded; receiving parties and purpose documented.

BIM Automation Starter Kit

A tool-agnostic pattern for auditing model health and compliance with Python. The example uses the open `ifcopenshell` library so it runs anywhere — adapt the same structure to the Revit API (`pyRevit`) or Navisworks where needed.

```
model_audit.py

# Lightweight BIM model-health & compliance audit.
# Tool-agnostic patterns — shown here with ifcopenshell (open source).
import re, csv
import ifcopenshell

NAMING = re.compile(r"^[A-Z]{2,6}-[A-Z]{2}-[A-Z0-9]{2,4}-[A-Z]{2}-\d{4}$")
REQUIRED = ["Classification", "Status", "Phase"]

def check_naming(model):
    # Flag any element whose Name breaks the agreed convention.
    return [e.GlobalId for e in model.by_type("IfcElement")
            if e.Name and not NAMING.match(e.Name)]

def audit_parameters(model):
    # Find elements missing any mandatory property.
    gaps = []
    for e in model.by_type("IfcElement"):
        psets = ifcopenshell.util.element.get_psets(e)
        present = {k for p in psets.values() for k in p}
        missing = [p for p in REQUIRED if p not in present]
        if missing: gaps.append((e.GlobalId, missing))
    return gaps

def clash_pairs(model, tol=0.01):
    # Naive bounding-box overlap pass — seed for a full clash run.
    elems = model.by_type("IfcElement")
    hits = []
    for i, a in enumerate(elems):
        for b in elems[i+1:]:
            if boxes_overlap(a, b, tol): hits.append((a.GlobalId, b.GlobalId))
    return hits

def write_report(rows, path="audit_report.csv"):
    with open(path, "w", newline="") as f:
        csv.writer(f).writerows([("id", "issue"), *rows])

if __name__ == "__main__":
    m = ifcopenshell.open("model.ifc")
    write_report([(g, "naming") for g in check_naming(m)] +
                [(g, "params:" + ",".join(x)) for g, x in audit_parameters(m)])
```

Wire it into your workflow

Validation only earns its keep when it runs automatically. The pattern below turns the audit script into a self-running quality gate on your Common Data Environment.

— The three-step automation loop

- ☐ **Trigger** — fire on every container uploaded to the CDE (Power Automate / webhook).
- ☐ **Validate** — run `model_audit.py` against naming, parameters and clashes.
- ☐ **Report** — post a pass/fail summary to Teams and log results to a Power BI model-health dashboard.

Why it matters: the same trigger → validate → report shape scales to any review workflow. Catch issues at upload — before they reach a coordination meeting, a client review, or an ISO 19650 audit — and your data becomes the most trusted source on the programme instead of a liability.

Want this running on your programme?

CK builds bespoke BIM automation, ISO 19650 workflows and digital-delivery strategy for infrastructure teams worldwide.

[Start a conversation →](#)